

MIPS32

INSTRUCTION SET ARCHITECTURE

1. Uvod u MIPS32 arhitekturu

Svaki procesor podržava određeni skup instrukcija, odnosno elementarnih operacija koje može izvršavati. Taj skup, zajedno sa pravilima kodiranja instrukcija, načinom adresiranja operanada te modelom interakcije sa registrima i memorijom, naziva se instruction set architecture (ISA).

ISA predstavlja osnovni interfejs između softvera i hardvera: kompajleri prevode programe napisane u jezicima visokog nivoa, kao što je C++, u niz mašinskih instrukcija koje procesor može direktno izvršavati.

MIPS (Microprocessor without Interlocked Pipeline Stages) nastao je početkom 1980-ih na Univerzitetu Stanford, a kasnije je komercijalno razvijan kroz kompaniju MIPS Technologies. Ova arhitektura pripada porodici RISC (Reduced Instruction Set Computer) procesora. Osnovna ideja RISC pristupa jeste pojednostavljenje skupa instrukcija i organizacije procesora: instrukcije su relativno jednostavne, pravilno oblikovane, jednake dužine, a pristup memoriji jasno je odvojen od računskih operacija.

U okviru MIPS32 arhitekture instrukcije su fiksne dužine od 32 bita, što dodatno olakšava njihovo dekodiranje i analizu.

1.1. Osnovne osobine i značaj MIPS arhitekture

MIPS arhitekturu karakterišu jednostavan i pregledan skup instrukcija, strogo definisan model registara i jasan load/store pristup radu sa memorijom. Takva organizacija omogućava relativno jednostavnu implementaciju procesora, efikasno dekodiranje instrukcija i pregledan model izvršavanja programa. Zbog tih osobina MIPS je imao značajnu ulogu u razvoju RISC procesora i predstavljao je jednu od važnijih referentnih arhitektura u širem razvoju savremenih procesorskih sistema.

Historijski posmatrano, MIPS procesori korišteni su u radnim stanicama i serverima kompanije SGI, u mrežnoj i telekomunikacijskoj opremi, modemima, štampačima i drugim ugradbenim sistemima. Pored toga, MIPS je bio prisutan i u više poznatih igračih platformi, uključujući Nintendo 64, PlayStation, PlayStation 2 i PlayStation Portable.

Iako MIPS danas nema onu tržišnu ulogu koju je imao u pojedinim ranijim periodima, arhitektura nije nestala iz prakse. I dalje se koristi u određenim ugradbenim i mikrokontrolerskim rješenjima, a jedan od primjera predstavljaju Microchip PIC32 mikrokontroleri zasnovani na MIPS32 jezgrama.

1.2. Load/store arhitektura

MIPS koristi load/store model, što znači da aritmetičke i logičke instrukcije rade isključivo nad registrima, a ne direktno nad memorijskim lokacijama. Ako je potrebno izvršiti operaciju nad podacima koji se nalaze u memoriji, ti podaci se najprije moraju učitati u registre pomoću instrukcija za učitavanje (load), zatim se nad njima izvršava željena operacija, a rezultat se po potrebi vraća u memoriju instrukcijama za pohranu (store). Takva organizacija pojednostavljuje implementaciju procesora, jer se računске jedinice mogu projektovati tako da rade nad malim i brzim skupom registara, dok se pristup sporijoj glavnoj memoriji odvaja u posebne instrukcije.

Drugim riječima, osnovni tok obrade podataka u MIPS arhitekturi može se posmatrati kao slijed: **memorija** → **registri** → **ALU** → **registri** → **memorija**. U tom smislu, registri predstavljaju neposredne operande računskih instrukcija, dok memorija služi kao veći, ali sporiji prostor za trajno ili privremeno čuvanje podataka.

1.3. Zašto su važni registri, memorija, grananja i procedure?

Za razumijevanje MIPS arhitekture od ključnog značaja su četiri međusobno povezana koncepta: registri, memorija, grananja i procedure:

- Registri predstavljaju najbrže mjesto za čuvanje i obradu podataka, jer se nalaze neposredno unutar procesora.
- Memorija, s druge strane, omogućava smještanje znatno veće količine podataka, ali uz veće kašnjenje pristupa.
- Grananja (branch instrukcije) omogućavaju realizaciju uslovnog toka izvršavanja, odnosno konstrukcija kao što su if, if/else i različite vrste petlji.
- Procedure, odnosno funkcije, omogućavaju modularnu organizaciju programa, ponovnu upotrebu koda i realizaciju rekurzivnih algoritama, ali istovremeno zahtijevaju pažljivo upravljanje stekom i registarskim stanjem.

Upravo zbog toga registri, memorija, grananja i procedure čine osnovu svakog realnog programa.

2. Registri i osnovni model izvršavanja

MIPS32 raspolaže sa 32 registra opšte namjene, pri čemu je svaki registar širok 32 bita. Pored njih, arhitektura uključuje i nekoliko specijalnih registara koji imaju posebnu ulogu u toku izvršavanja instrukcija.

2.1. 32 registra opšte namjene

Svaki od 32 registra opšte namjene ima svoj broj, od \$0 do \$31, kao i konvencionalno simboličko ime.

Važno je naglasiti da su, sa stanovišta hardvera, gotovo svi registri opšte namjene međusobno ravnopravni, odnosno da razlike u njihovoj upotrebi proizlaze prvenstveno iz softverskih konvencija koje su uspostavljene između kompajlera, operativnog sistema i programera. Najvažniji izuzetak predstavlja registar \$zero, čija je vrijednost hardverski fiksirana na nulu, tako da svaki pokušaj upisa u taj registar nema efekta.

Broj	Ime	Oznaka	Uloga	Caller/Callee saved
\$0	\$zero	r0	Uvijek sadrži vrijednost 0; upis u ovaj registar nema efekta.	N/A
\$1	\$at	r1	Assembler temporary; rezervisan za potrebe assemblera, naročito pri širenju pseudo-instrukcija.	N/A
\$2	\$v0	r2	Prva povratna vrijednost funkcije.	Caller-saved
\$3	\$v1	r3	Druga povratna vrijednost funkcije, odnosno dodatni registar za povrat rezultata kada je to potrebno.	Caller-saved

Broj	Ime	Oznaka	Uloga	Caller/Callee saved
\$4	\$a0	r4	Prvi argument funkcije.	Caller-saved
\$5	\$a1	r5	Drugi argument funkcije.	Caller-saved
\$6	\$a2	r6	Treći argument funkcije.	Caller-saved
\$7	\$a3	r7	Četvrti argument funkcije.	Caller-saved
\$8-15	\$t0-\$t7	r8-15	Privremeni registri (temporary), namijenjeni kratkotrajnom čuvanju međurezultata.	Caller-saved
\$16-23	\$s0-\$s7	r16-23	Sačuvani registri (saved); ako ih pozvana funkcija koristi, mora ih obnoviti prije povratka.	Callee-saved
\$24-25	\$t8-\$t9	r24-25	Dodatni privremeni registri.	Caller-saved
\$26-27	\$k0-\$k1	r26-27	Rezervisani za kernel i obradu izuzetaka; ne koriste se u korisničkim programima.	N/A
\$28	\$gp	r28	Global pointer; koristi se za pristup određenim globalnim podacima prema ABI konvenciji.	Callee-saved
\$29	\$sp	r29	Stack pointer; pokazuje na vrh steka.	Callee-saved
\$30	\$fp	r30	Frame pointer; baza tekućeg stack frame-a funkcije, kada se koristi.	Callee-saved
\$31	\$ra	r31	Return address; sadrži povratnu adresu koju upisuje instrukcija jal.	Caller-saved

2.2. Specijalni registri

Pored registara opšte namjene, MIPS arhitektura uključuje i nekoliko specijalnih registara sa posebnom funkcijom u toku izvršavanja programa.

- Registar PC (Program Counter) sadrži adresu instrukcije koja je na redu za izvršavanje, odnosno upravlja tokom izvođenja programa. Njegova vrijednost se uobičajeno ažurira nakon svake instrukcije, osim u slučajevima kada instrukcije grananja ili skoka eksplicitno promijene tok izvršavanja.
- Registri HI i LO koriste se za čuvanje rezultata određenih aritmetičkih operacija, prije svega množenja i dijeljenja.
Instrukcije kao što su *mult* i *multu* proizvode rezultat širine 64 bita, koji se smješta u par registara (HI, LO), pri čemu viši dio rezultata završava u registru HI, a niži dio u registru LO. Instrukcije *mfhi* i *mflo* služe za prenos sadržaja ovih registara u registre opšte namjene.

2.3. Caller-saved vs. Callee-saved

Jedna od najvažnijih konvencija u MIPS programiranju odnosi se na pitanje očuvanja registara prilikom poziva funkcija. Kada jedna funkcija poziva drugu, mora postojati jasan dogovor o tome koje registre po potrebi čuva pozivalac (**caller**), a koje mora očuvati pozvana funkcija (**callee**).

U grupu caller-saved registara spadaju registri \$t0-\$t9, registri za argumente \$a0-\$a3, registri za povratne vrijednosti \$v0-\$v1, kao i registar \$ra u smislu da ga funkcija koja dalje poziva druge procedure mora po potrebi sačuvati. Ako pozivalac želi da zadrži vrijednosti ovih registara i nakon funkcijskog poziva, tada ih mora sam prethodno pohraniti, najčešće na stek.

Nasuprot tome, registri \$s0-\$s7, kao i registri \$gp, \$sp i \$fp, pripadaju grupi callee-saved registara. To znači da funkcija koja ih koristi mora prije njihove izmjene sačuvati njihove prethodne vrijednosti, a prije povratka ih obnoviti. Na osnovu te konvencije pozivalac može računati na to da će sadržaj ovih registara ostati nepromijenjen nakon završetka pozvane funkcije.

Takva podjela registara omogućava dosljednu i efikasnu realizaciju funkcijskih poziva, posebno u složenijim programima sa većim brojem procedura, lokalnih varijabli i rekurzivnih poziva.

3. Pregled MIPS32 instrukcija

U cilju jasnijeg prikaza, MIPS32 instrukcije su u nastavku organizovane po funkcionalnim grupama. Ovakva klasifikacija omogućava lakše razumijevanje njihove uloge u arhitekturi procesora i u prevođenju konstrukcija iz jezika visokog nivoa u assembler.

3.1 Instrukcije za prenos podataka / Data Transfer

Budući da MIPS koristi load/store model, prijenos podataka između memorije i registara ostvaruje se isključivo pomoću posebnih instrukcija za učitavanje i pohranu podataka. Aritmetičke i logičke instrukcije ne mogu neposredno pristupati memoriji, nego operišu isključivo nad registrima. Zbog toga ova grupa instrukcija predstavlja osnovu cjelokupnog modela izvršavanja programa u MIPS arhitekturi: prije bilo kakve obrade podatke je potrebno dovesti u registre, a po završetku obrade rezultat se po potrebi vraća u memoriju.

3.1.1 Memorijski model i adresiranje

MIPS koristi **byte-addressable** memoriju, što znači da svaki bajt ima vlastitu adresu. Međutim, registri opšte namjene u MIPS32 arhitekturi široki su 32 bita, odnosno 4 bajta, pa se cjelobrojne vrijednosti tipa *word* uobičajeno posmatraju kao četverobajtnje cjeline. Pristup jednoj 32-bitnoj vrijednosti iz memorije zahtijeva da njena adresa bude pravilno poravnana (*aligned*) na granicu od 4 bajta, odnosno da bude djeljiva sa 4. Na sličan način, vrijednost tipa *halfword* zauzima 2 bajta i treba biti poravnana na adresu djeljivu sa 2.

Adresiranje memorijskih operanada u MIPS-u najčešće koristi oblik **offset(base)**, gdje je:

- **base** registar koji sadrži baznu adresu,
- **offset** 16-bitna konstantna vrijednost koja se sabira sa sadržajem baznog registra.

Na primjer, zapis **8(\$sp)** označava memorijsku adresu dobijenu sabiranjem sadržaja registra **\$sp** i konstante **8**.

Ovakav oblik adresiranja posebno je važan pri radu sa stekom, nizovima i strukturama podataka smještenim u memoriji.

3.1.2. Sign extension i zero extension

Kada se iz memorije učitava podatak manji od 32 bita, na primjer jedan bajt ili jedan *halfword*, potrebno je odrediti kako će se popuniti preostali, viši biti registra odredišta. U MIPS arhitekturi postoje dva osnovna pristupa:

- Kod **sign extension** pristupa, bit predznaka učitano podataka kopira se u više bite registra. Ako se, na primjer, učitava bajt, tada se bit 7 proširuje na sve više bite 32-bitnog registra;

ako se učitava *halfword*, tada se proširuje bit 15. Ovaj pristup koristi se kada se učitana vrijednost tumači kao označeni broj.

- Kod **zero extension** pristupa, svi viši biti registra popunjavaju se nulama, bez obzira na sadržaj najvišeg bita učitane podatka. Ovaj pristup koristi se kada se podatak posmatra kao neoznačena (*unsigned*) vrijednost. Zbog toga MIPS za učitavanje bajta i *halfword*-a nudi zasebne instrukcije za obje varijante.

3.1.3. Tabela instrukcija za prenos podataka

Instrukcija	Sintaksa	Opis	Napomena	Pseudo
lw	lw \$rt, offset(\$rs)	Učitava 32-bitnu word vrijednost iz memorije u registar \$rt.	Adresa treba biti poravnana na 4 bajta.	Ne
sw	sw \$rt, offset(\$rs)	Pohranjuje 32-bitnu word vrijednost iz registra \$rt u memoriju.	Adresa treba biti poravnana na 4 bajta.	Ne
lb	lb \$rt, offset(\$rs)	Učitava bajt iz memorije i proširuje ga na 32 bita pomoću sign extension pravila.	Bit 7 se proširuje na više bitove.	Ne
lbu	lbu \$rt, offset(\$rs)	Učitava bajt iz memorije i proširuje ga na 32 bita pomoću zero extension pravila.	Viši biti se popunjavaju nulama.	Ne
sb	sb \$rt, offset(\$rs)	Pohranjuje najniži bajt registra \$rt u memoriju.	Pohranjuje se samo 8 bita.	Ne
lh	lh \$rt, offset(\$rs)	Učitava 16-bitni halfword iz memorije i proširuje ga pomoću sign extension pravila.	Adresa treba biti poravnana na 2 bajta.	Ne
lhu	lhu \$rt, offset(\$rs)	Učitava 16-bitni halfword iz memorije i proširuje ga pomoću zero extension pravila.	Adresa treba biti poravnana na 2 bajta.	Ne
sh	sh \$rt, offset(\$rs)	Pohranjuje najnižih 16 bita registra \$rt u memoriju.	Pohranjuje se samo halfword.	Ne
lui	lui \$rt, imm	Učitava 16-bitnu konstantu u viši halfword registra \$rt, dok se niži biti postavljaju na nulu.	Često se koristi pri formiranju 32-bitnih konstanti ili adresa.	Ne
li	li \$rt, imm	Učitava konstantu u registar \$rt.	Pseudo-instrukcija; assembler je razvija u jednu ili više stvarnih instrukcija.	Da
la	la \$rt, labela	Učitava adresu labele u registar \$rt.	Pseudo-instrukcija; assembler koristi kombinaciju stvarnih instrukcija.	Da
move	move \$rt, \$rs	Kopira sadržaj registra \$rs u registar \$rt.	Pseudo-instrukcija; najčešće se razvija u addu \$rt, \$rs, \$zero.	Da

3.1.4. Pseudo-instrukcije u ovoj grupi

a) *li* – *Load Immediate*

Pseudo-instrukcija **li** koristi se za učitavanje konstante u registar. Na primjer, zapis:

li \$t0, 100

označava da se vrijednost 100 treba smjestiti u registar \$t0. Tačan način na koji će assembler razviti ovu pseudo-instrukciju zavisi od veličine konstante.

Za male konstante (koje stanu u 16 bita, tj. -32768 do 32767 za signed, 0 do 65535 za unsigned):

Razvoj li \$t0, 100 (mala konstanta)

```
addiu $t0, $zero, 100    # ili: ori $t0, $zero, 100
```

Za veće konstante (koje ne stanu u 16 bita), assembler koristi dvije instrukcije, npr. za:

li \$t0, 0x12345678

Razvoj li \$t0, 0x12345678 (velika konstanta)

```
lui  $t0, 0x1234    # gornjih 16 bita konstante
ori  $t0, $t0, 0x5678 # donjih 16 bita (OR, jer gornji su već postavljeni)
```

b) *la* – *Load Address*

Pseudo-instrukcija **la** koristi se za učitavanje adrese labele u registar. Na primjer, instrukcija:

la \$t0, niz

učitava adresu labele niz u registar \$t0.

Ovo je naročito važno pri radu sa globalnim varijablama, nizovima i drugim objektima smještenim u .data segmentu. Assembler pri tome najčešće koristi kombinaciju instrukcija kao što su lui i ori, odnosno addiu, u zavisnosti od konkretne lokacije podatka u memoriji.

c) *move* – *Kopiranje registra*

Pseudo-instrukcija **move** koristi se za kopiranje sadržaja jednog registra u drugi. Na primjer, zapis:

move \$t1, \$t0

znači da sadržaj registra \$t0 treba kopirati u \$t1.

Pošto MIPS nema posebnu osnovnu instrukciju samo za kopiranje registara, assembler ovu pseudo-instrukciju razvija u odgovarajuću stvarnu instrukciju, najčešće:

addu \$t1, \$t0, \$zero ili **ori \$t1, \$t0, 0**.

3.1.5. Primjeri – Data Transfer instrukcije

Primjer 1: Dodjela vrijednosti globalnoj varijabli

C++ kod:

C++

```
int x;           // globalna varijabla
int main() {
    x = 42;
    return 0;
}
```

MIPS kod (pretpostavka: x je u .data segmentu, \$t0 = privremeni registar):

```
MIPS
      .data
x:      .word    0          # rezerviši 4 bajta za globalnu varijablu x

      .text
main:
      li        $t0, 42     # $t0 = 42 (pseudo-instrukcija)
      la        $t1, x      # $t1 = adresa varijable x
      sw        $t0, 0($t1) # pohrani $t0 na adresu x (offset 0 od $t1)
```

Objašnjenje:

1. li \$t0, 42 – Assembler prevodi ovo u addiu \$t0, \$zero, 42.
Rezultat: registar \$t0 sada sadrži vrijednost 42.
2. la \$t1, x – Assembler učitava adresu labela x (npr. 0x10010000) u \$t1.
Ovo koristimo jer u MIPS-u ne možemo direktno reći 'pohrani na x', već moramo znati adresu.
3. sw \$t0, 0(\$t1) – Store word: pohrani 32 bita iz \$t0 na memorijsku adresu \$t1 + 0.
Ovo je jedini način da vrijednost iz registra dospije u memoriju.

Primjer 2: Učitavanje i kopiranje varijable

C++ kod:

```
C++
int a = 10;
int b;
b = a;    // kopiranje

MIPS
      .data
a:      .word    10
b:      .word    0

      .text
main:
      la        $t0, a      # $t0 = adresa a
      lw        $t1, 0($t0) # $t1 = vrijednost a (učitaj iz memorije)
      la        $t2, b      # $t2 = adresa b
      sw        $t1, 0($t2) # pohrani vrijednost a u b
```

Objašnjenje: lw (load word) čita 4 bajta iz memorije i smješta ih u registar. Adresa je \$t0 + 0 = adresa varijable a. Onda sw (store word) upisuje tu vrijednost na adresu varijable b. Dakle, **b = a** je realizovano kao: učitaj **a** u registar, pohrani registar u **b**.

Primjer 3: Pristup elementu niza

C++ kod:

C++	
<pre>int arr[5] = {10, 20, 30, 40, 50}; int x = arr[2]; // pristup trećem elementu (indeks 2)</pre>	
MIPS	
<pre> .data arr: .word 10, 20, 30, 40, 50 # niz od 5 int-ova .text main: la \$t0, arr # \$t0 = bazna adresa niza arr lw \$t1, 8(\$t0) # \$t1 = arr[2]; offset = 2 * 4 = 8 bajta # Sada \$t1 sadrži vrijednost 30</pre>	

Objašnjenje adresiranja niza: Svaki int zauzima 4 bajta.

Element arr[0] je na adresi arr+0, arr[1] na arr+4, arr[2] na arr+8 itd.

Opšta formula: adresa arr[i] = bazna_adresa + i * 4.

Primjer 4: Učitavanje bajta (lb i lbu)

C++	
<pre>char c = -5; // signed char: vrijednost -5 (0xFB u binarnom) int x = c; // sign extension: x = -5 unsigned int y = (unsigned char)c; // zero extension: y = 251</pre>	
MIPS	
<pre> .data c: .byte -5 # 0xFB .text main: la \$t0, c lb \$t1, 0(\$t0) # \$t1 = -5 (sign extended: 0xFFFFFFFFB) lbu \$t2, 0(\$t0) # \$t2 = 251 (zero extended: 0x000000FB)</pre>	

Ovo ilustruje razliku između lb i lbu.

Bajt 0xFB ima gornji bit postavljen na 1 (11111011).

Instrukcija lb radi sign extension – kopira bit 7 u sve gornje bite, što daje 0xFFFFFFFFB = -5 u dva-komplement kodu.

Instrukcija lbu popunjava gornje bite nulama: 0x000000FB = 251.

Isti bajt u memoriji, ali različita interpretacija!

3.1.6. Endianness: raspored bajta u memoriji

Kada se višebajtna vrijednost, kao što je 32-bitni word, smjesti u memoriju, postavlja se pitanje kojim redoslijedom se njeni bajti raspoređuju po adresama. Taj raspored naziva se endianness. Postoje dva osnovna pristupa:

- big-endian: najznačajniji bajt (MSB) nalazi se na nižoj memorijskoj adresi. Ovakav raspored se povezuje sa nekim MIPS konfiguracijama, klasičnom arhitekturom Motorola 68k, kao i sa mrežnim protokolima kao što je TCP/IP.
- little-endian: najmanje značajan bajt (LSB) nalazi se na nižoj memorijskoj adresi. Ovakav raspored koriste arhitekture x86, ARM u osnovnoj konfiguraciji, i RISC-V u osnovnoj konfiguraciji.

Primjer: Posmatrajmo 32-bitnu vrijednost: **0x12345678**

Ona se sastoji od četiri bajta:

Bajt 0 (MSB)	Bajt 1	Bajt 2	Bajt 3 (LSB)
0x12	0x34	0x56	0x78

Ako je početna adresa 0x10010000, raspored u memoriji izgleda ovako:

Adresa	Big-Endian	Little-Endian	
0x10010000	0x12	0x78	Najniža adresa
0x10010001	0x34	0x56	
0x10010002	0x56	0x34	
0x10010003	0x78	0x12	Najviša adresa

Važno je naglasiti da se numerička vrijednost u registru ne mijenja. Registar i dalje sadrži isti 32-bitni broj 0x12345678. Endianness utiče samo na to kako su bajti te vrijednosti raspoređeni u memoriji.

3.2. Aritmetičke instrukcije

Aritmetičke instrukcije predstavljaju osnovu svake računske obrade podataka. U MIPS arhitekturi ove instrukcije rade isključivo nad registrima, što je direktna posljedica load/store modela. U okviru ove grupe posebno je važno razlikovati operacije nad označenim (**signed**) i neoznačenim (**unsigned**) vrijednostima, kao i instrukcije koje pri prekoračenju opsega mogu izazvati izuzetak od onih koje takvo prekoračenje ignorišu.

3.2.1. Signed vs. Unsigned i Overflow

U MIPS32 arhitekturi označene cjelobrojne vrijednosti predstavljaju se u dva-komplement kodu. Kod aritmetičkih operacija nad takvim vrijednostima može doći do situacije u kojoj rezultat nije moguće ispravno predstaviti unutar 32 bita. Takva situacija naziva se overflow. Na primjer, ako se saberu dvije velike pozitivne 32-bitne označene vrijednosti, rezultat može izaći iz dozvoljenog opsega i dovesti do prekoračenja.

Zbog toga MIPS za pojedine operacije nudi dvije varijante instrukcija. Jedna varijanta tretira operaciju kao označene i može izazvati overflow izuzetak, dok druga varijanta ne generiše overflow izuzetak, nego zadržava samo donjih 32 bita rezultata.

Na primjer, instrukcija `add` može izazvati izuzetak pri signed overflow-u, dok instrukcija `addu` ne prijavljuje overflow, bez obzira na sadržaj operanada. Slično važi i za parove `sub/subu` te `addi/addiu`.

Važno je naglasiti da naziv `unsigned` u instrukcijama kao što su `addu` i `addiu` ne znači nužno da se operandi uvijek moraju tumačiti kao neoznačeni brojevi, nego prije svega da se pri izvršavanju ne generiše overflow izuzetak. Zbog toga se u praktičnom kodu i kompajlerski generisanom assembleru vrlo često koriste upravo ove varijante. Zato C++ kompajleri tipično generišu `addu` čak i za signed int operacije jer C++ ne zahtijeva detekciju overflowa za `int`

3.2.2 Registar HI i LO

Množenje dvaju 32-bitnih brojeva može dati rezultat od 64 bita. MIPS rješava ovo korištenjem registara HI i LO: gornjih 32 bita rezultata idu u HI, a donjih 32 bita u LO.

Dijeljenje također koristi HI i LO: LO dobija količnik, a HI ostatak. Instrukcije `mfhi` i `mflo` (Move From HI/LO) prenose te vrijednosti u opšte registre.

3.2.3. Tabela aritmetičkih instrukcija

Instrukcija	Sintaksa	Opis	Napomena	Pseudo
<code>add</code>	<code>add \$rd, \$rs, \$rt</code>	$\$rd = \$rs + \$rt$	Označeno sabiranje; može izazvati overflow izuzetak.	Ne
<code>addu</code>	<code>addu \$rd, \$rs, \$rt</code>	$\$rd = \$rs + \$rt$	Ne generiše overflow izuzetak; zadržava donjih 32 bita rezultata.	Ne
<code>addi</code>	<code>addi \$rt, \$rs, imm</code>	$\$rt = \$rs + \text{sign_ext}(imm)$	Neposredna konstanta je 16-bitna i proširuje se predznakom; može izazvati overflow izuzetak.	Ne
<code>addiu</code>	<code>addiu \$rt, \$rs, imm</code>	$\$rt = \$rs + \text{sign_ext}(imm)$	Ne generiše overflow izuzetak; često se koristi pri radu sa adresama i stekom.	Ne
<code>sub</code>	<code>sub \$rd, \$rs, \$rt</code>	$\$rd = \$rs - \$rt$	Označeno oduzimanje; može izazvati overflow izuzetak.	Ne
<code>subu</code>	<code>subu \$rd, \$rs, \$rt</code>	$\$rd = \$rs - \$rt$	Ne generiše overflow izuzetak.	Ne
<code>mult</code>	<code>mult \$rs, \$rt</code>	$HI:LO = \$rs \times \rt	Označeno množenje; rezultat se smješta u HI i LO.	Ne
<code>multu</code>	<code>multu \$rs, \$rt</code>	$HI:LO = \$rs \times \rt	Neoznačeno množenje; rezultat se smješta u HI i LO.	Ne
<code>div</code>	<code>div \$rs, \$rt</code>	$LO = \$rs \div \rt , $HI = \$rs \bmod \rt	Označeno dijeljenje; količnik ide u LO, ostatak u HI.	Ne
<code>divu</code>	<code>divu \$rs, \$rt</code>	$LO = \$rs \div \rt , $HI = \$rs \bmod \rt	Neoznačeno dijeljenje.	Ne
<code>mfhi</code>	<code>mfhi \$rd</code>	$\$rd = HI$	Prenosi sadržaj registra HI u registar opšte namjene.	Ne
<code>mflo</code>	<code>mflo \$rd</code>	$\$rd = LO$	Prenosi sadržaj registra LO u registar opšte namjene.	Ne
<code>mul</code>	<code>mul \$rd, \$rs, \$rt</code>	$\$rd = \text{nižih 32 bita proizvoda } \$rs \times \$rt$	Zavisi od MIPS varijante	Zavisi

Instrukcija `mul` zauzima posebno mjesto u ovoj grupi, jer njen status zavisi od konkretne verzije MIPS arhitekture i od korištenog alata. U novijim MIPS32 varijantama, kao što je MIPS32 Release 2, ona postoji kao stvarna instrukcija koja direktno vraća donjih 32 bita proizvoda u odredišni

registar. Međutim, u starijim implementacijama ista naredba je pseudo-instrukcija, koja se razvija u kombinaciju instrukcija mult i mflo.

3.2.4. Primjeri – Aritmetičke instrukcije

Primjer 1: Sabiranje i oduzimanje

C++

```
int a = 5, b = 3;
int c = a + b;    // c = 8
int d = a - b;    // d = 2
```

Pretpostavka: \$s0 = a, \$s1 = b, \$s2 = c, \$s3 = d.

MIPS

```
li    $s0, 5      # $s0 = a = 5
li    $s1, 3      # $s1 = b = 3
addu  $s2, $s0, $s1 # $s2 = a + b = 8 (c)
subu  $s3, $s0, $s1 # $s3 = a - b = 2 (d)
```

Koristimo addu i subu jer C++ ne garantuje detekciju overflowa za int operacije tako da kompajleri mogu generisati unsigned varijante da izbjegnu nepotrebne iznimke.

Primjer 2: Izraz sa više operanada

C++

```
int a = 10, b = 4, c = 2;
int result = (a + b) * c;
```

Pretpostavka: \$s0 = a, \$s1 = b, \$s2 = c, \$t0 = privremeni, \$s3 = result.

MIPS

```
li    $s0, 10
li    $s1, 4
li    $s2, 2
addu  $t0, $s0, $s1 # $t0 = a + b = 14 (međurezultat)
mult  $t0, $s2      # HI:LO = 14 * 2 = 28
mflo  $s3           # $s3 = donja 32 bita = 28 (result)
```

Međurezultat (a+b) se čuva u \$t0. Zatim mult množi \$t0 i \$s2 i rezultat ide u HI:LO, ne direktno u registar. Pošto je rezultat mali (28), on je u LO, pa mflo prenosi LO u \$s3.

Primjer 3: Djeljenje i ostatak

C++

```
int a = 17, b = 5;
int q = a / b;    // q = 3
int r = a % b;    // r = 2
```

MIPS

```

li      $s0, 17
li      $s1, 5
div     $s0, $s1    # LO = 17/5 = 3, HI = 17%5 = 2
mflo    $s2         # $s2 = q = 3 (količnik)
mfhi    $s3         # $s3 = r = 2 (ostatak)

```

div ne daje direktno registar za rezultat, sve ide u HI i LO. mflo daje količnik, mfhi daje ostatak.

Primjer 4: Inkrementiranje i dekrementiranje

C++

```

int i = 0;
i++;      // i = 1
i--;      // i = 0

```

MIPS

```

li      $s0, 0
addiu   $s0, $s0, 1    # i++ : i = i + 1
addiu   $s0, $s0, -1   # i-- : i = i + (-1) = i - 1

```

addiu \$s0, \$s0, 1 je najčešći način za inkrementiranje. addiu koristi 16-bitnu signed konstantu (immediate), tako da -1 radi savršeno za dekrementiranje.

3.3. Logičke ALU instrukcije

U sistemskom programiranju i pri komunikaciji sa hardverskim uređajima logičke instrukcije imaju izuzetno važnu ulogu. Pomoću njih moguće je testirati da li je određeni bit postavljen, postaviti pojedine bite bez promjene ostalih, obrisati odabrane bite ili invertovati njihov sadržaj. Na taj način logičke operacije predstavljaju osnovni alat za preciznu kontrolu binarnog zapisa podataka.

3.3.1. Tabela logičkih instrukcija

Instrukcija	Sintaksa	Opis	Napomena	Pseudo
and	and \$rd, \$rs, \$rt	\$rd = \$rs AND \$rt		Ne
andi	andi \$rt, \$rs, imm	\$rt = \$rs AND zero_ext(imm)	Neposredna vrijednost se proširuje nulama (zero extension).	Ne
or	or \$rd, \$rs, \$rt	\$rd = \$rs OR \$rt		Ne
ori	ori \$rt, \$rs, imm	\$rt = \$rs OR zero_ext(imm)	Neposredna vrijednost se proširuje nulama.	Ne
xor	xor \$rd, \$rs, \$rt	\$rd = \$rs XOR \$rt		Ne
xori	xori \$rt, \$rs, imm	\$rt = \$rs XOR zero_ext(imm)	Neposredna vrijednost se proširuje nulama.	Ne
nor	nor \$rd, \$rs, \$rt	\$rd = NOT(\$rs OR \$rt)	Može se koristiti i za realizaciju operacije negacije nad jednim registrom.	Ne

3.4. Shift i bit manipulacija

Shift instrukcije pomjeraju bite unutar registra ulijevo ili udesno za određeni broj pozicija.

3.4.1. Logički vs. Aritmetički shift

U MIPS arhitekturi razlikuju se logički i aritmetički pomaci.

Kod logičkog pomjeranja, upražnjeni biti uvijek se popunjavaju nulama. To važi i za pomjeranje ulijevo (sll) i za logičko pomjeranje udesno (srl). Takav pristup je prirodan kada se operandi tumače kao neoznačene binarne vrijednosti.

Aritmetičko pomjeranje udesno (sra) razlikuje se po tome što upražnjene više bite popunjava kopijom bita predznaka, odnosno najvišeg bita registra. Na taj način se čuva znak broja pri pomjeranju udesno, što je posebno važno kod označenih vrijednosti.

Zbog toga logički desni shift odgovara dijeljenju neoznačene vrijednosti stepenima broja 2, dok aritmetički desni shift odgovara dijeljenju označenih vrijednosti stepenima broja 2, uz očuvanje predznaka.

3.4.2. Tabela shift instrukcija

Instrukcija	Sintaksa	Opis	Napomena	Pseudo
sll	sll \$rd, \$rt, shamt	$\$rd = \$rt \ll \text{shamt}$	Logički pomak ulijevo; shamt je 5-bitna konstanta iz opsega 0–31. Odgovara množenju sa 2^{shamt} .	Ne
srl	srl \$rd, \$rt, shamt	$\$rd = \$rt \gg \text{shamt}$	Logički pomak udesno; upražnjeni biti popunjavaju se nulama.	Ne
sra	sra \$rd, \$rt, shamt	$\$rd = \$rt \gg \text{shamt}$	Aritmetički pomak udesno; upražnjeni biti popunjavaju se bitom predznaka.	Ne
sllv	sllv \$rd, \$rt, \$rs	$\$rd = \$rt \ll \$rs$	Varijabilni logički pomak ulijevo; broj pozicija zadat je u registru \$rs. Koristi se donjih 5 bita registra \$rs.	Ne
srlv	srlv \$rd, \$rt, \$rs	$\$rd = \$rt \gg \$rs$	Varijabilni logički pomak udesno.	Ne
srav	srav \$rd, \$rt, \$rs	$\$rd = \$rt \gg \$rs$	Varijabilni aritmetički pomak udesno.	Ne

3.4.3. Primjeri – Shift instrukcije

Primjer 1: Brzo množenje i dijeljenje stepenima 2

C++

```
int x = 5;
int a = x * 4;    // = 20: kompajler može koristiti sll
int b = x / 4;    // = 1: kompajler može koristiti sra
unsigned int c = (unsigned)x / 4;  // = 1: srl
```

MIPS

```
li      $s0, 5
sll     $s1, $s0, 2    # $s1 = 5 << 2 = 20 (množenje s 4 = 2^2)
sra     $s2, $s0, 2    # $s2 = 5 >> 2 = 1 (dijeljenje signed s 4)
srl     $s3, $s0, 2    # $s3 = 5 >> 2 = 1 (dijeljenje unsigned s 4)
```

3.5. Poređenje i grananje

Instrukcije poređenja i grananja omogućavaju realizaciju uslovnog toka izvršavanja programa, odnosno konstrukcija kao što su if, if/else, while, for i do/while.

Za razliku od nekih drugih arhitektura, kao što je x86, koje koriste poseban skup statusnih zastavica (flags) za evidentiranje rezultata poređenja, MIPS uglavnom koristi ili direktno poređenje registara instrukcijama grananja beq i bne, ili posebne instrukcije poređenja koje rezultat upisuju u registar opšte namjene. Na taj način rezultat uslova postaje eksplicitno vidljiv u registrima, što dodatno naglašava jednostavnost i pravilnost MIPS modela izvršavanja.

Posebno važnu ulogu imaju instrukcije iz porodice **slt** (set less than), koje omogućavaju formiranje logičkog rezultata poređenja u registru. Takav rezultat zatim može biti korišten zajedno sa instrukcijama **beq** i **bne** za realizaciju složenijih uslova grananja.

3.5.1. Tabela instrukcija poređenja i grananja

Instrukcija	Sintaksa	Opis	Napomena	Pseudo
slt	slt \$rd, \$rs, \$rt	\$rd = 1 ako je \$rs < \$rt, inače \$rd = 0	Označeno (signed) poređenje.	Ne
sltu	sltu \$rd, \$rs, \$rt	\$rd = 1 ako je \$rs < \$rt, inače \$rd = 0	Neoznačeno (unsigned) poređenje.	Ne
slti	slti \$rt, \$rs, imm	\$rt = 1 ako je \$rs < sign_ext(imm), inače \$rt = 0	Označeno poređenje sa neposrednom vrijednošću.	Ne
sltiu	sltiu \$rt, \$rs, imm	\$rt = 1 ako je \$rs < sign_ext(imm), inače \$rt = 0	Poređenje je neoznačeno, ali se neposredna vrijednost najprije proširuje predznakom.	Ne
beq	beq \$rs, \$rt, label	Grananje na label ako je \$rs == \$rt	Branch on Equal.	Ne
bne	bne \$rs, \$rt, label	Grananje na label ako je \$rs != \$rt	Branch on Not Equal.	Ne
blt	blt \$rs, \$rt, label	Grananje ako je \$rs < \$rt	Pseudo-instrukcija; tipično se razvija pomoću slt i bne.	Da
bgt	bgt \$rs, \$rt, label	Grananje ako je \$rs > \$rt	Pseudo-instrukcija; tipično se razvija pomoću slt i bne.	Da
ble	ble \$rs, \$rt, label	Grananje ako je \$rs <= \$rt	Pseudo-instrukcija; tipično se razvija pomoću slt i beq.	Da
bge	bge \$rs, \$rt, label	Grananje ako je \$rs >= \$rt	Pseudo-instrukcija; tipično se razvija pomoću slt i beq.	Da
beqz	beqz \$rs, label	Grananje ako je \$rs == 0	Pseudo-instrukcija; ekvivalent beq \$rs, \$zero, label.	Da
bnez	bnez \$rs, label	Grananje ako je \$rs != 0	Pseudo-instrukcija; ekvivalent bne \$rs, \$zero, label.	Da

3.5.2. Relacija između poređenja i grananja

Instrukcije beq i bne omogućavaju direktno grananje na osnovu jednakosti ili nejednakosti sadržaja dva registra.

Međutim, za relacijske uslove kao što su “manje od”, “veće od”, “manje ili jednako” i “veće ili jednako” MIPS u klasičnom obliku koristi kombinaciju instrukcije poređenja i instrukcije grananja.

Na primjer, uslov oblika $\$rs < \rt često se realizuje tako što instrukcija `slt` najprije u određeni registar upiše vrijednost 1 ili 0, a zatim `bne` ili `beq` provjerava taj rezultat i odlučuje da li će doći do grananja. Ovakav pristup odražava osnovnu filozofiju MIPS arhitekture: složenije operacije se grade iz manjeg broja jednostavnih i jasno definisanih instrukcija.

3.5.3. Branch delay slot

Kod klasičnih MIPS procesora grananja i skokovi povezani su sa pojmom **branch delay slot**.

To znači da se instrukcija koja se nalazi neposredno iza branch ili jump instrukcije izvršava bez obzira na to da li je grananje zaista uzeto. Drugim riječima, efekat skoka nastupa tek nakon izvršavanja još jedne naredne instrukcije.

U ovom predavanju pretpostavlja se varijanta bez aktivnog delay slot ponašanja, osim ako nije eksplicitno navedeno drugačije.

3.5.4. NOP instrukcija

Instrukcija **NOP** znači **no operation**, odnosno „ne radi ništa“. Njena svrha je da zauzme jedno mjesto u toku izvršavanja, ali bez promjene sadržaja registara ili memorije.

U MIPS okruženju, `nop` se najčešće posmatra kao pseudo-instrukcija, a u praksi se tipično prevodi u: **`slil $zero, $zero, 0`**

Pošto se registar `$zero` uvijek čita kao 0, a upis u njega nema efekta, ova instrukcija ne mijenja stanje procesora i zato se ponaša kao „prazna“ operacija.

3.5.4. Primjeri – Grananje

Primjer 1: Prosta if naredba

C++

```
int a = 5, b = 10;
if (a < b) {
    a = a + 1;
}
```

Pretpostavka: `$s0 = a`, `$s1 = b`.

MIPS

```
li      $s0, 5
li      $s1, 10
# if (a < b): skačemo UNUTAR if-bloka ako je uvjet istinit
# Strategija: granaaj na 'end_if' ako uvjet NIJE
istinit(inverzija!)
slt     $t0, $s0, $s1 # $t0 = 1 ako a < b, inače 0
beq     $t0, $zero, end_if # ako $t0==0 (a >= b), preskoči
tijelo
addiu   $s0, $s0, 1    # a = a + 1 (tijelo if-a)
end_if:
# nastavak programa
```

slt \$t0, \$s0, \$s1 postavlja \$t0 na 1 ako $a < b$. Tada beq \$t0, \$zero, end_if – ako $t0 == 0$, što znači $a \geq b$ (uvjet nije ispunjen), preskačemo tijelo. Ako $t0 == 1$ ($a < b$, uvjet JE ispunjen), ne skačemo i izvršavamo tijelo.

Primjer 2: if/else naredba

C++

```
int a = 7, b = 3;
int max;
if (a > b) {
    max = a;
} else {
    max = b;
}
```

Pretpostavka: \$s0 = a, \$s1 = b, \$s2 = max.

MIPS

```
li    $s0, 7
li    $s1, 3
# if (a > b): ako a > b, idemo u 'then', inače 'else'
slt   $t0, $s1, $s0 # $t0 = 1 ako b < a (tj. a > b)
beq   $t0, $zero, else_branch # ako a <= b, idi u else
then_branch:
    addu $s2, $s0, $zero # max = a
    j    end_if
else_branch:
    addu $s2, $s1, $zero # max = b
end_if:
    # $s2 = max
```

Za provjeru $a > b$ koristimo: slt \$t0, \$s1, \$s0 provjerava $b < a$, što je ekvivalentno $a > b$.
U then grani: addu \$s2, \$s0, \$zero kopira a u max, a zatim j (jump) preskače else granu.
U else grani: addu \$s2, \$s1, \$zero kopira b u max. Label end_if označava nastavak programa.

Primjer 3: while petlja

C++

```
int i = 0, sum = 0;
while (i < 10) {
    sum = sum + i;
    i++;
}
```

Pretpostavka: \$s0 = i, \$s1 = sum.

MIPS

```
li    $s0, 0    # i = 0
li    $s1, 0    # sum = 0
li    $t9, 10   # konstanta 10 (za poređenje)
while_start:
    slt   $t0, $s0, $t9 # $t0 = 1 ako i < 10
    beq   $t0, $zero, while_end # ako i >= 10, izlaz
```

```

        addu    $s1, $s1, $s0    # sum = sum + i
        addiu   $s0, $s0, 1      # i++
        j       while_start     # nazad na provjeru uvjeta
while_end:
        # $s1 = 0+1+2+...+9 = 45

```

Struktura while petlje: provjera uvjeta → tijelo → bezuvjetni skok na provjeru → labela za izlaz. Provjera mora biti na početku. Ako uvjet nije ispunjen pri prvoj provjeri, tijelo se ne izvršava ni jedanput, upravo kao while petlja u C++.

Primjer 4: for petlja

C++

```

int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += i * i;
}

```

Pretpostavka: \$s0 = i, \$s1 = sum.

MIPS

```

        li      $s0, 0          # i = 0 (inicijalizacija for-a)
        li      $s1, 0          # sum = 0
for_start:
        slti    $t0, $s0, 5     # $t0 = 1 ako i < 5 (uvjet for-a)
        beq     $t0, $zero, for_end
        mult    $s0, $s0        # HI:LO = i * i
        mflo    $t1             # $t1 = i^2
        addu    $s1, $s1, $t1    # sum += i^2
        addiu   $s0, $s0, 1     # i++ (korak for-a)
        j       for_start
for_end:

```

for petlja se direktno preslikava na while: inicijalizacija ide ispred petlje, uvjet na početku petlje, korak na kraju tijela.

slti \$t0, \$s0, 5 je efikasnije od slt s registrom jer nema potrebe za zasebnim registrom za konstantu 5.

3.6. Instrukcije skoka i upravljanja tokom izvršavanja (Jump & control flow)

Instrukcije skoka imaju ključnu ulogu u upravljanju tokom izvršavanja programa. Za razliku od instrukcija uslovnog grananja, koje zavise od ispunjenosti određenog uslova i koriste relativno adresiranje u odnosu na tekuću poziciju programa, jump instrukcije omogućavaju bezuslovnu promjenu toka izvršavanja. Na taj način koriste se za prelazak na drugi dio programa, zatvaranje petlji, preskakanje pojedinih grana izvršavanja, kao i za realizaciju poziva i povratka iz funkcija.

Posebno su značajne instrukcije jal i jr, jer predstavljaju osnovu mehanizma poziva funkcije u MIPS arhitekturi.

- Instrukcija **jal** omogućava prelazak na adresu funkcije i istovremeno čuva povratnu adresu.
- Instrukcija **jr** omogućava povratak na prethodno sačuvanu lokaciju.

3.6.1. Tabela jump instrukcija

Instrukcija	Sintaksa	Opis	Napomena	Pseudo
j	j label	Bezuslovni skok na zadanu labelu.	Ciljna adresa formira se iz 26-bitnog polja instrukcije i dijela tekuće adrese.	Ne
jal	jal label	Poziv funkcije; čuva povratnu adresu i skače na zadanu labelu.	Povratna adresa se upisuje u registar \$ra.	Ne
jr	jr \$rs	Skok na adresu zapisanu u registru \$rs.	Najčešće se koristi za povratak iz funkcije u obliku jr \$ra.	Ne
jalr	jalr \$rd, \$rs	Poziv funkcije na adresi zapisanoj u registru \$rs, uz čuvanje povratne adrese u \$rd.	Ako se \$rd izostavi, povratna adresa se uobičajeno upisuje u \$ra.	Ne

4. Procedure

U MIPS arhitekturi poziv funkcije nije skriven iza složenih hardverskih mehanizama, nego se realizuje pomoću precizno definisanih instrukcija, registara i konvencija.

Posebno važnu ulogu u tom procesu ima **calling convention**, odnosno skup pravila i dogovora koji određuju kako različiti dijelovi programa međusobno komuniciraju prilikom poziva funkcija. Ova pravila omogućavaju da funkcije napisane u različitim dijelovima programa, pa čak i funkcije dobijene različitim prevodiocima ili iz različitih biblioteka, mogu međusobno pravilno sarađivati bez konflikta u korištenju registara i steka.

4.1. Anatomija poziva funkcije

Poziv funkcije u MIPS-u može se posmatrati kao niz jasno definisanih koraka. Iako konkretan kod zavisi od složenosti funkcije, osnovni obrazac ostaje isti.

Prvo, funkcija koja poziva drugu funkciju, odnosno pozivalac (**caller**), priprema argumente. Prema uobičajenoj MIPS konvenciji, prva četiri argumenta smještaju se u registre \$a0-\$a3. Ako funkcija prima više od četiri argumenta, dodatni argumenti se uobičajeno pohranjuju na stek.

Nakon pripreme argumenata, pozivalac koristi instrukciju `jal` kako bi prebacio kontrolu izvršavanja na pozvanu funkciju. Instrukcija `jal` pri tome obavlja dvije operacije: skače na početak funkcije i istovremeno u registar \$ra upisuje povratnu adresu, odnosno adresu na koju se program treba vratiti nakon završetka pozvane procedure.

Po ulasku u funkciju, pozvana funkcija, odnosno **callee**, po potrebi izvršava **prolog**. Prolog funkcije obuhvata čuvanje registara koje funkcija mora očuvati, rezervaciju prostora na steku za lokalne varijable i, po potrebi, postavljanje dodatnih pomoćnih pokazivača kao što je \$fp. Nakon toga slijedi tijelo funkcije, u kojem se izvršavaju potrebne operacije.

Po završetku rada funkcije, rezultat se najčešće smješta u registar \$v0, a kada je potrebno vratiti dodatnu vrijednost ili širi rezultat, može se koristiti i \$v1. Prije povratka funkcija izvršava **epilog**, u okviru kojeg obnavlja sačuvane registre i vraća pokazivač steka u prethodno stanje. Na kraju se instrukcijom `jr $ra` kontrola vraća nazad pozivaocu, koji zatim može preuzeti rezultat iz registra \$v0.

Ukratko, ti koraci su opisani u tabeli:

Korak	Opis
Priprema argumenata	Pozivalac (<i>caller</i>) smješta prva četiri argumenta u registre \$a0-\$a3. Ako funkcija prima više od četiri argumenta, dodatni argumenti se pohranjuju na stek.
Poziv funkcije	Pozivalac izvršava instrukciju <code>jal</code> , koja prebacuje kontrolu izvršavanja na pozvanu funkciju i istovremeno u registar \$ra upisuje povratnu adresu.
Prolog funkcije	Pozvana funkcija (<i>callee</i>), po potrebi, čuva registre koje mora očuvati, rezerviše prostor na steku za lokalne varijable i eventualno postavlja registar \$fp.
Tijelo funkcije	Funkcija izvršava potrebne operacije i koristi registre i memoriju u skladu sa calling convention pravilima.
Smještanje rezultata	Povratna vrijednost funkcije najčešće se smješta u registar \$v0, a po potrebi i u \$v1.
Epilog funkcije	Pozvana funkcija obnavlja sačuvane registre i vraća pokazivač steka \$sp u prethodno stanje.

Korak	Opis
Povratak pozivaocu	Instrukcijom <code>jr \$ra</code> kontrola izvršavanja vraća se na povratnu adresu koju je ranije upisala instrukcija <code>jal</code> .
Preuzimanje rezultata	Nakon povratka iz funkcije, pozivalac preuzima rezultat iz registra <code>\$v0</code> i nastavlja izvršavanje programa.

4.2. Uloga registara pri pozivu funkcije

Poziv funkcije u MIPS-u zasniva se na jasno definisanim ulogama pojedinih registara. Registri `$a0-$a3` koriste se za proslijeđivanje prvih argumenata funkcije, dok registri `$v0-$v1` služe za povratne vrijednosti. Registar `$ra` čuva povratnu adresu koju upisuje instrukcija `jal`, a registri `$sp` i eventualno `$fp` koriste se za upravljanje stekom i lokalnim kontekstom funkcije.

4.3. Prolog i epilog funkcije

Prolog i epilog predstavljaju standardni obrazac ulaska u funkciju i izlaska iz nje. Njihova svrha je očuvanje konteksta funkcije i pravilna organizacija steka.

U prologu funkcija najprije rezerviše prostor na steku tako što umanjuje vrijednost registra `$sp`. Nakon toga po potrebi pohranjuje registre koje mora očuvati, kao što su `$s0-$s7`, a u funkcijama koje dalje pozivaju druge funkcije čuva se i `$ra`. Ako funkcija koristi lokalne varijable koje se ne mogu smjestiti isključivo u registre, i za njih se u prologu rezerviše prostor na steku.

U epilogu se izvodi suprotan postupak: sačuvani registri se obnavljaju, prostor na steku se oslobađa vraćanjem `$sp` na prethodnu vrijednost, a zatim se izvršava `jr $ra` radi povratka pozivaocu.

Takva organizacija čini izvršavanje funkcija predvidivim i omogućava pravilno funkcionisanje i kod složenijih slučajeva, uključujući ugniježdene i rekurzivne pozive.

4.4. Stack frame

Svaka funkcija koja koristi stek tipično formira vlastiti stack frame, odnosno dio steka koji pripada upravo toj aktivaciji funkcije. U tom okviru mogu se nalaziti sačuvani registri, lokalne varijable, dodatni argumenti koji nisu stali u registre, kao i eventualni pomoćni podaci potrebni funkciji.

U MIPS arhitekturi stek uobičajeno raste prema nižim adresama. To znači da se pri rezervaciji prostora na steku vrijednost registra `$sp` smanjuje, a pri oslobađanju prostora povećava.

Na primjer, instrukcija:

`addiu $sp, $sp, -N`

rezerviše `N` bajta prostora na steku, dok instrukcija

`addiu $sp, $sp, N`

oslobađa prethodno rezervisani prostor.

Registar `$sp` (**stack pointer**) uvijek pokazuje na vrh tekućeg steka, odnosno na najnovije rezervisani dio memorije. Pored njega, u nekim funkcijama koristi se i registar `$fp` (**frame pointer**), koji može služiti kao stabilna referentna tačka unutar stack frame-a. To je naročito korisno kada funkcija ima više lokalnih varijabli ili kada koristi promjenljiv broj argumenata. U jednostavnijim funkcijama kompajler može izostaviti upotrebu `$fp` i koristiti samo `$sp`.

4.5. Leaf i non-leaf funkcije

U analizi MIPS procedura korisno je razlikovati leaf i non-leaf funkcije:

- Leaf funkcija je funkcija koja ne poziva nijednu drugu funkciju. Pošto u takvom slučaju ne dolazi do prepisivanja registra \$ra novim jal pozivom, leaf funkcija ne mora čuvati \$ra na stek.
- Nasuprot tome, non-leaf funkcija poziva barem jednu drugu funkciju. U tom slučaju novi jal poziv prepisuje sadržaj registra \$ra, pa funkcija prije daljeg pozivanja mora sačuvati prethodnu povratnu adresu na stek. Zbog toga su non-leaf funkcije tipično složenije i gotovo uvijek zahtijevaju eksplicitno upravljanje stack frame-om.

4.5.1 Rekurzija i značaj steka

Poseban slučaj funkcijskih poziva predstavlja rekurzija, odnosno situacija u kojoj funkcija poziva samu sebe.

U takvim slučajevima stek postaje apsolutno neophodan, jer svaka aktivacija funkcije mora imati vlastiti skup sačuvanih registara, vlastite argumente i vlastiti povratni kontekst. Bez steka bi se pri svakom novom rekurzivnom pozivu prepisivale prethodne vrijednosti, čime bi korektno izvršavanje bilo onemogućeno.

4.6. Primjeri – Procedure

Primjer 1: Prosta funkcija s argumentima i povratnom vrijednošću

C++

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int result = add(3, 7); // result = 10  
}
```

MIPS

```
main:  
    li      $a0, 3          # 1. argument: a = 3  
    li      $a1, 7          # 2. argument: b = 7  
    jal     add             # pozovi add(); $ra = sljedeća ins.  
    addu    $s0, $v0, $zero # $s0 = result = 10 (povratna vr. u $v0)  
    # nastavak...  
  
add:  
    # Prolog: ova funkcija ne poziva dalje i ne koristi $s, pa nema  
    # stek alokacije  
    addu    $v0, $a0, $a1   # $v0 = a + b  
    jr      $ra             # povratak, $v0 nosi rezultat
```

Ovo je leaf funkcija jer ne poziva dalje nijednu drugu funkciju. Zato ne mora čuvati \$ra. Argumenti dolaze u \$a0 i \$a1. Rezultat se smješta u \$v0 i caller ga čita odatle. Funkcija nema lokalne varijable pa ne treba ni stack frame.

Primjer 2: Funkcija koja poziva drugu

C++

```
int square(int x) { return x * x; }

int sum_of_squares(int a, int b) {
    return square(a) + square(b);
}
```

MIPS

```
square:
    mult    $a0, $a0        # HI:LO = x * x
    mflo    $v0             # $v0 = x^2
    jr      $ra

sum_of_squares:
    # Prolog: sačuvati $ra i $a1 (jal prepíše $ra, a $a1 nam treba)
    addiu   $sp, $sp, -12   # alociraj 12 bajta na steku
    sw      $ra, 8($sp)     # sačuvaj $ra
    sw      $s0, 4($sp)     # sačuvaj $s0 (callee-saved)
    sw      $a1, 0($sp)     # sačuvaj b ($a0/$a1 su caller-saved!)
    # Poziv square(a)
    jal     square          # $a0 = a (već tu), $ra = povratna adr.
    addu    $s0, $v0, #zero  # $s0 = square(a)
    # Poziv square(b)
    lw      $a0, 0($sp)     # $a0 = b (učitaj sačuvano b)
    jal     square
    addu    $v0, $s0, $v0    # $v0 = square(a) + square(b)
    # Epilog: obnovi registre
    lw      $ra, 8($sp)
    lw      $s0, 4($sp)
    addiu   $sp, $sp, 12    # dealociraj stack frame
    jr      $ra
```

Ovaj primjer pokazuje sve elemente upravljanja stekom.

Kada `sum_of_squares` poziva `square`, `jal` prepíše `$ra`. Zato moramo pohraniti `$ra` na stek PRIJE `jal`.

Koristimo `$s0` za čuvanje međurezultata (`square(a)`) jer `$s` registri su callee-saved pa moramo ih sačuvati. `b` je bio u `$a1` koji je caller-saved i može biti prepisan pozivom, pa i njega čuvamo.

Primjer 3: Rekurzivna funkcija (Fibonacci)

C++

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

MIPS

```
fib:
    # Prolog
    addiu   $sp, $sp, -12
    sw      $ra, 8($sp)
```

```

sw      $s0, 4($sp)    # $s0 = n
sw      $s1, 0($sp)    # $s1 = fib(n-1)
addu    $s0, $a0, $zero # $s0 = n
# if (n <= 1) return n
slti    $t0, $s0, 2    # $t0 = 1 ako n < 2 (tj. n <= 1)
beq     $t0, $zero, fib_recurse
addu    $v0, $s0, $zero # return n
j       fib_epilog

fib_recurse:
addiu   $a0, $s0, -1   # a0 = n-1
jal     fib
addu    $s1, $v0, $zero # $s1 = fib(n-1)
addiu   $a0, $s0, -2   # a0 = n-2
jal     fib
addu    $v0, $s1, $v0  # $v0 = fib(n-1) + fib(n-2)

fib_epilog:
lw      $ra, 8($sp)
lw      $s0, 4($sp)
lw      $s1, 0($sp)
addiu   $sp, $sp, 12
jr      $ra

```

Rekurzija zahtijeva da svaki poziv funkcije ima vlastiti prostor na steku za povratnu adresu, sačuvane registre i lokalne podatke. U ovom primjeru svaki poziv funkcije fib alocira 12 bajta na steku. Pri povratku iz rekurzije stek se postepeno “odmotava”, a prethodno sačuvane vrijednosti se obnavljaju. Bez pravilnog upravljanja stekom, rekurzivni pozivi bi prepisivali stanje prethodnih poziva i funkcija ne bi mogla raditi ispravno.

5. Kodiranje instrukcija – R, I i J format

Jedna od osnovnih osobina MIPS32 arhitekture jeste činjenica da je svaka instrukcija fiksne dužine od 32 bita, odnosno 4 bajta. Ova osobina predstavlja jedno od karakterističnih obilježja RISC arhitektura, jer značajno pojednostavljuje dekodiranje instrukcija u procesoru.

Za razliku od arhitektura sa promjenljivom dužinom instrukcija, kod kojih procesor mora najprije odrediti granice i dužinu instrukcije, u MIPS-u se svaka instrukcija posmatra kao jedinstvena 32-bitna riječ fiksnog oblika. Time se pojednostavljuje hardverska realizacija.

MIPS32 definiše tri osnovna formata instrukcija:

1. R-format (register format),
2. I-format (immediate format) i
3. J-format (jump format).

Iako su sve instrukcije jednake ukupne dužine, raspored polja unutar tih 32 bita zavisi od toga koje informacije instrukcija treba sadržavati. Na primjer, instrukcije koje rade nad registrima zahtijevaju polja za specifikaciju registara, dok instrukcije grananja, učitavanja ili skoka zahtijevaju polja za neposredne vrijednosti ili ciljnu adresu.

5.1. R-format (Register format)

R-format koriste instrukcije koje operišu isključivo nad registrima. U ovu grupu spadaju, na primjer, instrukcije add, sub, and, or, slt i mnoge druge.

Ovaj format sadrži polja za tri registra, dodatno polje za pomak (shift amount) i završno polje funct, koje precizno određuje konkretnu operaciju.

Kod R-formata polje opcode ima vrijednost 000000, pa se stvarna vrsta instrukcije određuje pomoću polja funct.



Primjer: Kodiranje instrukcije **add \$s0, \$t1, \$t2** (add rd=s0, rs=t1, rt=t2):

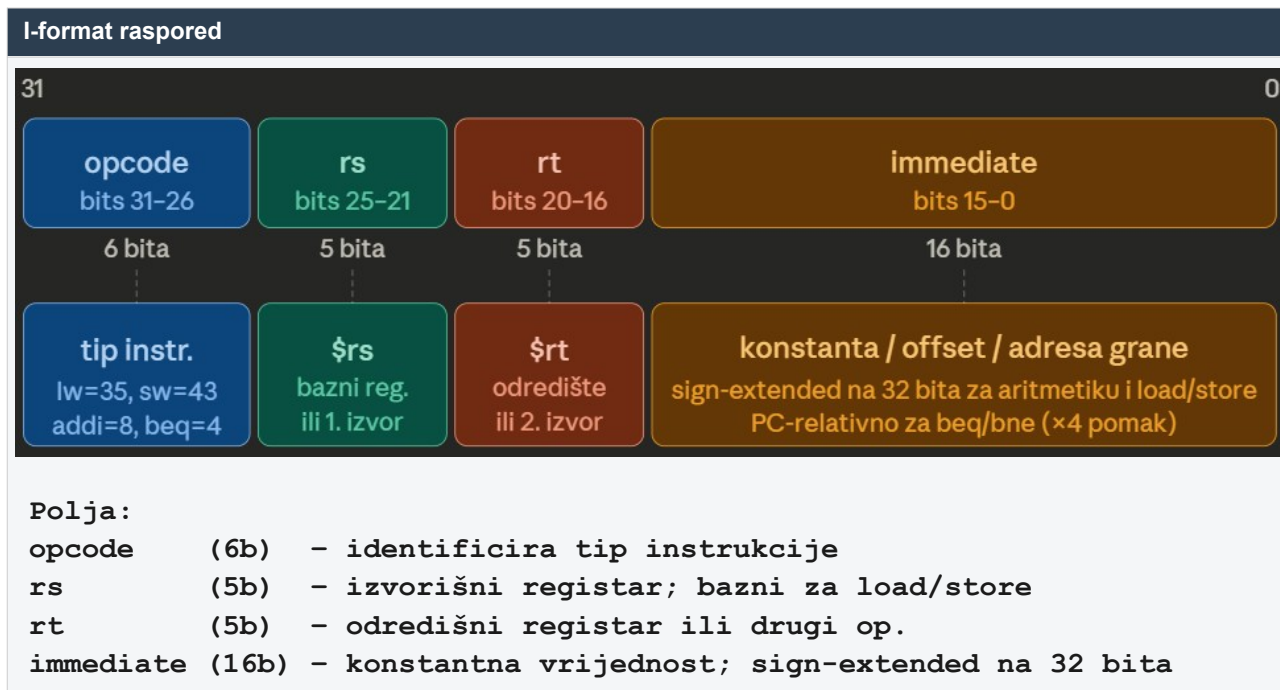
Polje	Vrijednost (binarno)	Vrijednost (decimalno)	Objašnjenje
opcode	000000	0	R-format uvijek 0
rs	01001	9	\$t1 = registar 9
rt	01010	10	\$t2 = registar 10
rd	10000	16	\$s0 = registar 16
shamt	00000	0	Nema shifta
funct	100000	32	add funct kod = 32

Cijela instrukcija u binarnom zapisu: 000000 01001 01010 10000 00000 100000

U heksadecimalnom zapisu: 0x012A8020. Ovo je tačno ono što procesor vidi u memoriji na adresi instrukcije.

5.2. I-format (Immediate format)

I-format koriste instrukcije koje, pored registara, sadrže i neposrednu vrijednost (immediate). U ovu grupu spadaju instrukcije kao što su `addi`, `addiu`, `lw`, `sw`, `beq`, `bne`, `slti`, `andi`, `ori`, `xori`, `lui` i druge. Za razliku od R-formata, ovdje ne postoji posebno polje `rd`; odredišni registar je, kada postoji, smješten u polju `rt`.



Primjer: Kodiranje `addi $t0, $s0, 100` (`addi rt=$t0, rs=$s0, imm=100`):

Polje	Vrijednost (binarno)	Vrijednost (decimalno)	Objašnjenje
opcode	001000	8	addi opcode = 8
rs	10000	16	\$s0 = registar 16
rt	01000	8	\$t0 = registar 8
immediate	0000000001100100	100	Konstanta 100 u 16 bita

Binarno: 001000 10000 01000 0000000001100100 → Hex: 0x22080064

Primjer: Kodiranje `lw $t1, 8($sp)` (load word, `rt=$t1, rs=$sp, offset=8`):

Polje	Vrijednost (binarno)	Vrijednost (decimalno)	Objašnjenje
opcode	100011	35	lw opcode = 35
rs	11101	29	\$sp = registar 29
rt	01001	9	\$t1 = registar 9
immediate	0000000000001000	8	Offset = 8 bajtova

Kod load/store instrukcija polje `rs` označava bazni registar, `rt` označava registar u koji se učitava podatak ili iz kojeg se pohranjuje podatak, a `immediate` predstavlja pomak u bajtima u odnosu na baznu adresu.

Primjer: Branch instrukcija i PC-relativno adresiranje

Instrukcije uslovnog grananja, kao što su beq i bne, koriste PC-relativno adresiranje. To znači da se u njihovom immediate polju ne nalazi apsolutna adresa ciljne instrukcije, nego pomak (offset) u odnosu na adresu instrukcije koja slijedi nakon branch instrukcije, odnosno u odnosu na PC + 4.

Važno je naglasiti da se ovaj pomak izražava u broju instrukcija, a ne direktno u bajtovima. Razlog za to je činjenica da su MIPS instrukcije uvijek poravnane na 4 bajta, pa se pri formiranju stvarne ciljne adrese efektivno koristi pomak pomnožen sa 4.

PC-relativno adresiranje za branch

```
# Primjer:
0x00400000: beq $t0, $t1, labela # ako $t0 == $t1, skoči na labelu
0x00400004: addi $t0, $t0, 1          # ovo se izvršava ako branch nije uzet
0x00400008: j end
0x0040000C: labela: add $s0, $s0, $t0

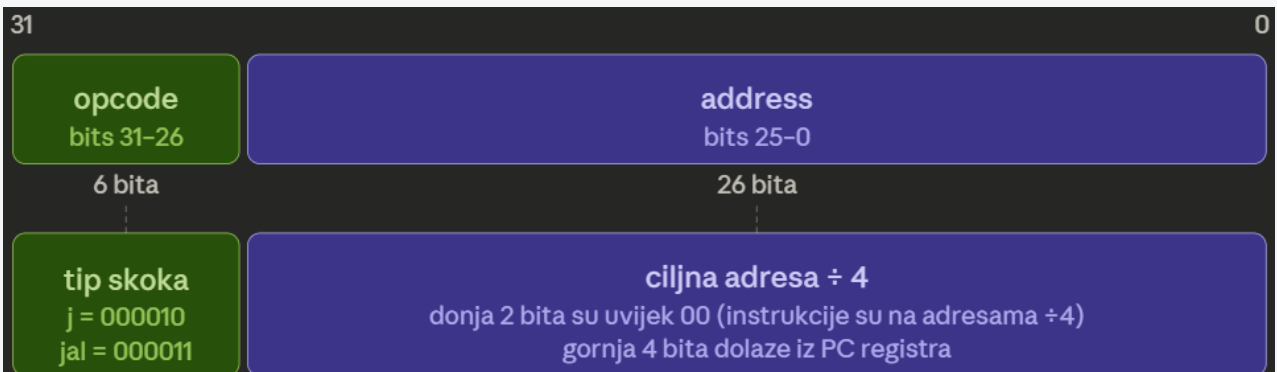
# Offset za branch = (adresa_labela - (PC+4)) / 4
#                   = (0x0040000C - 0x00400004) / 4 = 0x00000008 / 4 = 2
# Immediate polje = 2 (16-bitna signed vrijednost = 0000000000000010)

# Kodiranje beq $t0, $t1, labela:
# opcode=000100, rs=$t0=01000, rt=$t1=01001, imm=0000000000000010
# 000100 01000 01001 0000000000000010 = 0x11090002
```

5.3. J-format (Jump format)

J-format koriste instrukcije j i jal. Za razliku od prethodna dva formata, ovdje se ne kodiraju registri, nego se koristi 26-bitno polje koje služi za formiranje ciljne adrese skoka.

J-format layout



Polja:
opcode (6b) - j = 000010 (2), jal = 000011 (3)
target (26b) - gornji 26 bita ciljne adrese podijeljeni s 4
(jer su instrukcije uvijek na adresama djeljivim s 4,
donja 2 bita su uvijek 00 i ne trebaju se kodirati)

Primjer: Kodiranje j 0x00400100:

Instrukcije j i jal ne sadrže punu 32-bitnu ciljnu adresu, nego samo 26-bitno target polje koje se pri izvršavanju kombinuje sa gornja 4 bita iz PC+4. Zbog toga one mogu skakati samo unutar istog 256 MB bloka adresnog prostora. Kada je potrebno izvršavanje prebaciti na proizvoljnu 32-bitnu adresu, ciljna adresa se najprije smješta u registar, a zatim se koristi instrukcija jr ili jalr.

6. Assembler direktive

Assembler direktive nisu MIPS instrukcije koje procesor izvršava, nego upute assembleru o tome kako treba organizovati kod, podatke, simbole i memorijski raspored programa. Zbog toga direktive ne predstavljaju izvršni kod u užem smislu, iako neke od njih rezervišu prostor u memoriji ili ga popunjavaju konkretnim vrijednostima. Bez razumijevanja assembler direktiva nije moguće napisati potpun program, jer se upravo pomoću njih definišu podaci, stringovi, početne vrijednosti i raspored sadržaja u memoriji.

6.1. Pregled najvažnijih direktiva

Tačan skup podržanih assembler direktiva i njihovo ponašanje značajno zavise od konkretnog assemblera, simulatora ili razvojnog okruženja. Zbog toga tabelu u nastavku treba posmatrati kao opšti pregled, a ne kao univerzalno pravilo za sve alate.

Direktiva	Primjer upotrebe	Opis
.org	.org 0x80020000	Postavlja adresu na koju će biti smješten naredni kod ili podaci.
.byte	.byte 42	Rezerviše 1 bajt i inicijalizira ga zadatom vrijednošću.
.word	.word 4200	Rezerviše 4 bajta i inicijalizira ih zadatom vrijednošću.
.ascii	.ascii "Hello"	Pohranjuje niz ASCII znakova bez završnog nul-bajta.
.asciz	.asciz "Hello"	Pohranjuje ASCII string sa završnim nul-bajtom.
.set / .equ	.equ SIZE, 10	Definiše simboličku konstantu.
.data	.data	Uobičajena segmentna direktiva.
.text	.text	Uobičajena direktiva za kodni segment.
.globl	.globl _start	Proglašava simbol globalnim.
.end	.end _start	Označava kraj modula.

6.1.1. Direktive za podatke

.word – cijeli brojevi i nizovi

Primjeri .word			
x:	.word	100	# int x = 100; (4 bajta)
arr:	.word	10, 20, 30, 40, 50	# int arr[5] = {10,20,30,40,50};
zero:	.word	0	# int zero = 0;

Labela ispred direktive (npr. arr:) daje simbolično ime toj memorijskoj lokaciji.

.space – rezervacija bez inicijalizacije

Primjeri .space

```
.data
buffer: .space 100      # char buffer[100]; (100 bajta, svi nula)
niz:    .space 40       # int niz[10]; (10 int-ova = 40 bajta)
```

Direktiva `.space` koristi se za rezervaciju određenog broja bajta u memoriji. Ova direktiva je naročito korisna kada je potrebno unaprijed obezbijediti prostor za nizove ili bafer-e, bez navođenja pojedinačnih početnih vrijednosti za svaki element.

.ascii i .asciiiz – stringovi

Primjeri .ascii i .asciiiz

```
.data
msg1: .asciiiz "Hello, world!" # string s null-terminatorom ('\0')
msg2: .ascii  "MIPS"          # string BEZ null-terminatora (opasno!)
newline: .asciiiz "\n"        # karakter novog reda
```

6.2. Kompletan primjer programa s direktivama

Ovaj program demonstrira kompletan tok jednostavne obrade podataka u QtMips okruženju: učitavanje podataka iz memorije, izvođenje aritmetičke operacije, pretvaranje numeričkog rezultata u ASCII zapis i ispis rezultata na standardni izlaz pomoću sistemskih poziva.

Na početku programa, koristeći Linux-stil `write syscall (4004)`, ispisuje se tekstualna poruka "Suma: ". Nakon toga program učitava dvije cjelobrojne vrijednosti `a` i `b` iz memorije, sabira ih i rezultat smješta u registar `$s2`.

Pošto QtMips ne podržava pojednostavljeni `syscall` za direktan ispis cijelog broja, rezultat se najprije mora pretvoriti u tekstualni oblik. To se postiže dijeljenjem sume sa 10, pri čemu količnik predstavlja cifru desetica, a ostatak cifru jedinica. Obje cifre se zatim pretvaraju u odgovarajuće ASCII znakove dodavanjem vrijednosti 48, koja odgovara znaku '0'. Tako dobijeni znakovi smještaju se u pomoćni bafer, nakon čega se i oni ispisuju pomoću `write syscall-a`. Na kraju se ispisuje znak novog reda i program se završava sistemskim pozivom `exit (4001)`.

Kompletan MIPS program

```
.org      0x80020000      # početna adresa programa
.text
_start:                                     # ulazna tačka programa

        addi    $v0, $zero, 4004           # syscall write
        addi    $a0, $zero, 1             # stdout
        la      $a1, msg                  # adresa stringa "Suma: "
        addi    $a2, $zero, 6             # dužina stringa
        syscall                                     # ispis stringa

        la      $t0, a                   # adresa varijable a
        lw      $s0, 0($t0)              # učitaj a

        la      $t0, b                   # adresa varijable b
        lw      $s1, 0($t0)              # učitaj b
```

```

        addu    $s2, $s0, $s1        # s2 = a + b

        addi    $t1, $zero, 10       # djelilac 10
        divu    $s2, $t1             # suma / 10
        mflo    $t2                  # desetice
        mfhi    $t3                  # jedinice

        addi    $t2, $t2, 48         # desetice -> ASCII
        addi    $t3, $t3, 48         # jedinice -> ASCII

        la      $t4, buf             # adresa bafera
        sb      $t2, 0($t4)          # upiši prvu cifru
        sb      $t3, 1($t4)          # upiši drugu cifru

print_number:
        addi    $v0, $zero, 4004     # syscall write
        addi    $a0, $zero, 1       # stdout
        addu    $a1, $t4, $zero      # adresa bafera
        addi    $a2, $zero, 2       # ispis 2 znaka
        syscall                                # ispis broja

print_newline:
        addi    $v0, $zero, 4004     # syscall write
        addi    $a0, $zero, 1       # stdout
        la      $a1, newline         # adresa "\n"
        addi    $a2, $zero, 1       # dužina = 1
        syscall                                # novi red

        addi    $v0, $zero, 4001     # syscall exit
        addi    $a0, $zero, 0       # status = 0
        syscall                                # izlaz iz programa

        .end _start                  # kraj programa

        .data                        # segment podataka
a:        .word    5                  # a = 5
b:        .word    11                # b = 11
msg:      .asciz   "Suma: "          # tekst za ispis
newline:  .asciz   "\n"              # novi red
buf:      .word    0                  # bafer za dvije ASCII cifre

```

7. Adresni prostor u MIPS arhitekturi

Razumijevanje memorijskog rasporeda predstavlja jedan od ključnih preduslova za pravilno razumijevanje rada MIPS procesora, naročito u kontekstu adresiranja, privilegovanih režima rada, upravljanja izuzecima i prevođenja virtuelnih adresa u fizičke. Iako se u jednostavnijim nastavnim primjerima memorija često prikazuje samo kroz segmente kao što su *text*, *data*, *heap* i *stack*, na nivou same MIPS arhitekture situacija je složenija. MIPS procesor radi sa jasno definisanim segmentima virtuelnog adresnog prostora, pri čemu se ti segmenti razlikuju prema načinu mapiranja, privilegijama pristupa i keširanju.

U MIPS32 okruženju procesor raspolaže **32-bitnim virtuelnim adresnim prostorom**, što znači da program vidi ukupno **4 GB adresnog prostora**. Taj prostor nije jedinstven i homogen, nego je podijeljen na više segmenata sa različitim svojstvima. Osnovna ideja takve organizacije jeste da se

jasno razdvoje korisnički i kernel prostor, kao i da se omogući fleksibilno mapiranje memorije putem TLB-a ili fiksnih mapiranja, zavisno od stanja sistema i konfiguracije MMU-a.

7.1. Režimi rada procesora

MIPS procesor može raditi u najmanje dva osnovna režima: **kernel mode** i **user mode**. U kernel modu procesor ima potpuni pristup memorijskom prostoru i privilegovanim instrukcijama. To znači da operativni sistem, exception handler-i i boot kod mogu pristupati i korisničkim i kernel adresama, kao i upravljati TLB-om, keš memorijom i kontrolnim registrima.

Nasuprot tome, u user mode režimu pristup memoriji i instrukcijama je ograničen. Programi u korisničkom režimu mogu koristiti samo one adrese i instrukcije koje su dozvoljene korisničkom procesu. Upravo zbog toga memorijska mapa nije samo pitanje organizacije podataka, nego i osnovni mehanizam zaštite sistema.

7.2. Osnovne osobine MIPS adresnog prostora

Segmenti MIPS virtuelnog adresnog prostora razlikuju se prema tri osnovne osobine:

- da li je pristup memoriji **mapiran** ili **nemapiran**
- da li je segment dostupan u **user** ili samo u **kernel** režimu
- da li se pristupi tom segmentu obavljaju kao **cached** ili **uncached**

Ako je segment **mapiran**, to znači da se virtuelna adresa mora prevesti u fizičku putem **TLB-a** (*Translation Lookaside Buffer*). Ako je segment **nemapiran**, fizička adresa se dobija direktno, bez TLB prevođenja.

MIPS32 pokriva 4 gigabajta fizičke memorije. Taj prostor je podijeljen na segmente čiji nazivi potiču iz MIPS terminologije:

Segment	Virtualne adrese	Veličina	Osnovne osobine	Tipična namjena
kseg3	0xE000_0000 – 0xFFFF_FFFF	0.5 GB	Kernel segment, uobičajeno cacheable , koristi TLB-mapiranje	Viši dio kernel virtuelnog adresnog prostora; koristi se za potrebe operativnog sistema i drugih privilegovanih komponenti
kseg2	0xC000_0000 – 0xDFFF_FFFF	0.5 GB	Kernel segment, uobičajeno cacheable , koristi TLB-mapiranje	Dio višeg kernel adresnog prostora; historijski povezan sa <i>supervisor mode</i> režimom
kseg1	0xA000_0000 – 0xBFFF_FFFF	0.5 GB	Kernel segment, uncached , direktno mapiran na istih donjih 512 MB fizičke memorije kao kseg0	Boot kod, pristup ulazno-izlaznim uređajima i drugim memorijski mapiranim resursima
kseg0	0x8000_0000 – 0x9FFF_FFFF	0.5 GB	Kernel segment, cacheable , direktno mapiran na donjih 512 MB fizičke memorije	Brz pristup memoriji iz privilegovanog režima, bez TLB prevođenja
kuseg	0x0000_0000 – 0x7FFF_FFFF	2 GB	Korisnički segment, dostupan i kernelu, uobičajeno cacheable , koristi TLB-mapiranje	Korisnički programi i njihove logičke oblasti, uključujući izvršni kod, globalne podatke, <i>heap</i> i <i>stack</i>

7.3. Cached i uncached pristup memoriji

Jedna od važnijih ideja u MIPS memorijskoj organizaciji jeste razlika između **cacheable** i **uncached** segmenata.

Kada je segment cacheable, pristup memoriji može koristiti keš hijerarhiju procesora, što ubrzava izvršavanje programa. Kada je segment uncached, svaki pristup ide direktno prema memoriji ili uređaju.

To je naročito važno u dva slučaja:

- tokom **boot procesa**, kada cache još nije inicijalizovan
- pri radu sa **memory-mapped I/O** uređajima, gdje se ne smije koristiti zastarjela vrijednost iz cache-a

Zbog toga kseg1 ima posebnu praktičnu ulogu jer omogućava pouzdan pristup memoriji i uređajima bez zavisnosti od stanja keša.

7.4. Fiksno mapiranje memorije

Pored klasičnog TLB-modela, određene MIPS konfiguracije podržavaju i **fixed mapping** pristup. U takvom režimu prevođenje memorije nije dinamičko, nego je unaprijed određeno. To znači da određeni virtuelni segmenti uvijek odgovaraju unaprijed zadatim fizičkim oblastima.

Ovakav pristup pojednostavljuje rad sistema u specifičnim okruženjima, naročito kada nije potrebna puna fleksibilnost virtuelne memorije. Ipak, osnovna ideja ostaje ista: različiti segmenti imaju različitu namjenu i različita pravila pristupa.

7.5. Memorijska mapa pri pokretanju sistema

Poseban slučaj predstavlja memorijska mapa tokom **reset-a**, odnosno prije nego što su TLB i keš inicijalizirani. U toj fazi procesor koristi poseban boot raspored memorije. Boot kod mora biti smješten u segmentu **kseg1**, upravo zato što je taj segment uncached i direktno mapiran.

Posebno je značajna adresa **0xBFC00000**, koja predstavlja podrazumijevani **boot exception vector**. To je adresa sa koje procesor počinje dohvat instrukcija pri uključivanju napajanja ili reset-u. Pošto se nalazi u kseg1 segmentu, pristup toj adresi ne zavisi od TLB-a niti od keš memorije. Time se osigurava da procesor uvijek ima pouzdanu početnu tačku izvršavanja.

8. Logički raspored korisničkog programa

Uobičajeni prikaz korisničkog programa u MIPS okruženju zasniva se na četiri osnovne logičke oblasti:

- **text segment**
- **data segment**
- **heap**
- **stack**

Važno je naglasiti da ove oblasti ne predstavljaju zasebne MIPS arhitekturne segmente, nego uobičajeni raspored jednog korisničkog programa unutar korisničkog dijela adresnog prostora, odnosno unutar segmenta **kuseg**.

8.1. Tipična memorijska mapa

Adresni prostor korisničkog programa često se opisuje na sljedeći način:

Oblast	Približni adresni opseg	Sadržaj	Ko pristupa
Kernel	0x80000000 – 0xFFFFFFFF	Kernel prostor, exception handler-i, sistemske strukture, uređaji	Operativni sistem / privilegovani režim
Stack segment	vrh oko 0x7FFFFFFF, raste nadole	Lokalne varijable funkcija, stack frame-ovi, sačuvani registri, povratne adrese	Program preko registra \$sp
Heap segment	približno od 0x10040000, raste nagore	Dinamički alocirani podaci	Program preko mehanizma za alokaciju memorije
Data segment	približno 0x10010000 – 0x10040000	Globalne i statičke varijable, nizovi, stringovi	Program preko labela i lw/sw pristupa
Text segment	približno od 0x00400000	Izvršni kod programa	PC, j, jal, grananja
Niske rezervisane adrese	donji dio adresnog prostora	Null adresa i druge posebne lokacije	Ne koristiti u korisničkom programu

Tačne početne adrese i granice pojedinih oblasti mogu zavisi od konkretnog simulatora, linkera, ABI konvencije ili operativnog sistema.

8.2. Odnos između kuseg segmenta i oblasti text, data, heap i stack

Potrebno je razlikovati arhitekturne MIPS segmente od logičkih oblasti korisničkog programa. Oblasti text, data, heap i stack nisu zasebni MIPS arhitekturni segmenti na istom nivou kao kuseg ili kseg0-kseg3. One samo predstavljaju logičke cjeline rasporeda korisničkog programa koje se najčešće nalaze unutar segmenta kuseg.

Drugim riječima:

- **kuseg** je arhitekturni korisnički segment MIPS32 virtuelnog adresnog prostora
- **text, data, heap i stack** predstavljaju tipične logičke oblasti rasporeda programa unutar tog prostora

8.2.1 Heap

Heap segment služi za dinamičku alokaciju memorije. Za razliku od globalnih podataka, koji su poznati unaprijed, heap se koristi kada je memorijski prostor potrebno rezervisati tokom izvršavanja programa. Heap uobičajeno raste prema višim adresama, pa se zato kaže da heap “raste nagore”.

8.2.2. Stack

Stack segment koristi se za čuvanje lokalnog konteksta funkcija. U njemu se tipično nalaze:

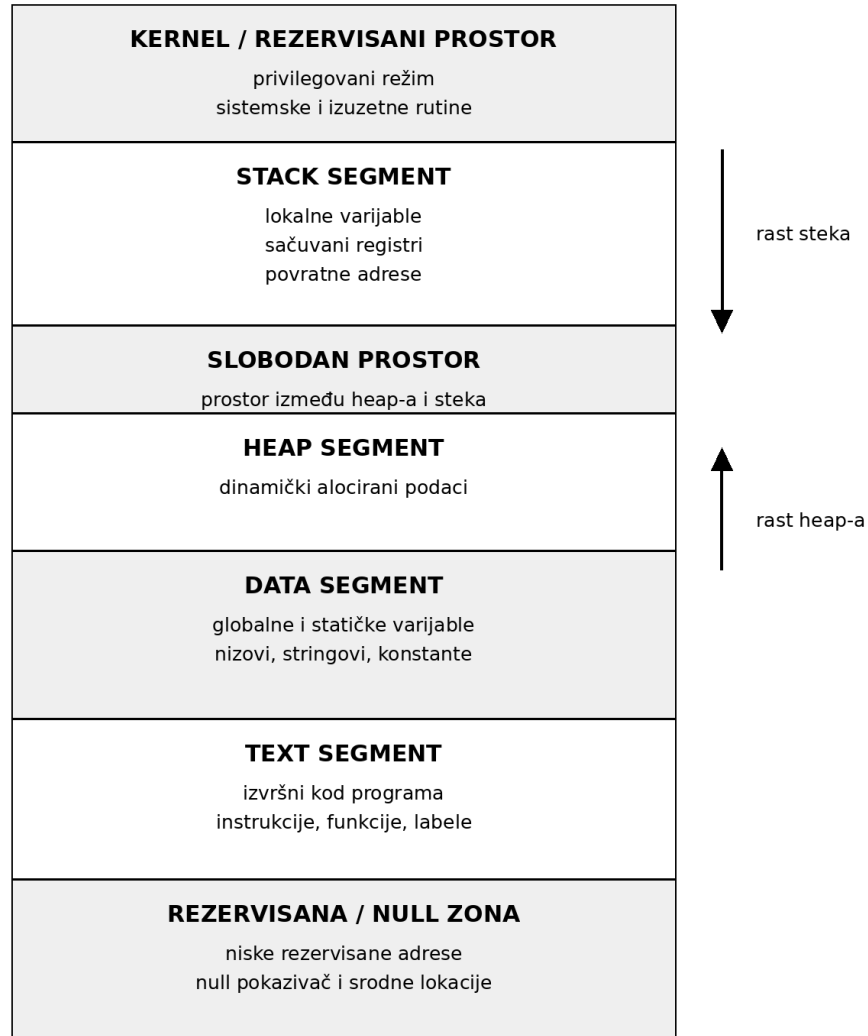
- lokalne varijable
- sačuvani registri
- povratne adrese

- dodatni argumenti funkcija
- drugi privremeni podaci

Stek raste prema nižim adresama, pa se pri rezervaciji prostora registar \$sp smanjuje, a pri oslobađanju povećava.

Na slici ispod je dat opšti prikaz MIPS memorijskog rasporeda.

Visoke adrese



Niske adrese

9. Primjeri pisanja MIPS asemblerskog koda

Z1. Suma elemenata niza

Napisati funkciju koja prima niz cijelih brojeva i njegovu dužinu, te vraća sumu svih elemenata.

C++

```
int sumArray(int* arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    return sum;
}
```

Mapiranje registara: \$a0 = arr (bazna adresa niza), \$a1 = n (broj elemenata), \$v0 = sum (povratna vrijednost), \$t0 = i (indeks), \$t1 = privremeni (tekući element).

MIPS

```
sumArray:
    # Prolog (leaf funkcija, nema stack frame)
    li    $v0, 0          # sum = 0
    li    $t0, 0          # i = 0

sum_loop:
    slt    $t2, $t0, $a1  # $t2 = 1 ako i < n
    beq    $t2, $zero, sum_done # ako i >= n, izlaz

    sll    $t1, $t0, 2     # $t1 = i * 4 (offset u bajtima)
    addu   $t1, $a0, $t1   # $t1 = adresa arr[i] = arr + i*4
    lw     $t1, 0($t1)     # $t1 = arr[i] (učitaj element)

    addu   $v0, $v0, $t1   # sum += arr[i]
    addiu  $t0, $t0, 1     # i++
    j      sum_loop

sum_done:
    jr     $ra            # return sum (već u $v0)
```

Z2. Najveći element niza

Pronaći najveći element u nizu cijelih brojeva. Pretpostavljamo $n \geq 1$.

C++

```
int findMax(int* arr, int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

Mapiranje: \$a0 = arr, \$a1 = n, \$v0 = max, \$t0 = i, \$t1 = arr[i].

MIPS

```
findMax:
    lw      $v0, 0($a0)    # max = arr[0]
    li      $t0, 1         # i = 1
max_loop:
    slt     $t2, $t0, $a1  # i < n?
    beq     $t2, $zero, max_done

    sll     $t1, $t0, 2     # offset = i * 4
    addu    $t1, $a0, $t1   # adresa arr[i]
    lw      $t1, 0($t1)    # $t1 = arr[i]

    slt     $t2, $v0, $t1   # $t2 = 1 ako max < arr[i]
    beq     $t2, $zero, skip_update # ako arr[i] <= max, preskoči

    addu    $v0, $t1, $zero # max = arr[i]

skip_update:
    addiu   $t0, $t0, 1     # i++
    j       max_loop

max_done:
    jr      $ra
```

Z3. Bubble Sort

Sortirati niz cijelih brojeva u rastućem redoslijedu pomoću bubble sort-a.

C++

```
void bubbleSort(int* arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

Mapiranje: \$a0 = arr, \$a1 = n, \$s0 = i, \$s1 = j, \$t0 = arr[j], \$t1 = arr[j+1], \$t2 = adresa arr[j], \$t3 = adresa arr[j+1].

MIPS

```
bubbleSort:
    addiu    $sp, $sp, -8    # sačuvaj callee-saved
    sw       $s0, 4($sp)
    sw       $s1, 0($sp)

    li       $s0, 0          # i = 0

outer_loop:
    addiu    $t9, $a1, -1    # $t9 = n-1
    slt      $t8, $s0, $t9   # i < n-1?
    beq      $t8, $zero, outer_done

    li       $s1, 0          # j = 0

inner_loop:
    subu     $t9, $a1, $s0   # $t9 = n - i
    addiu    $t9, $t9, -1    # $t9 = n - 1 - i
    slt      $t8, $s1, $t9   # j < n-1-i?
    beq      $t8, $zero, inner_done

    # Izračunaj adrese arr[j] i arr[j+1]
    sll      $t2, $s1, 2     # offset j*4
    addu     $t2, $a0, $t2   # adresa arr[j]
    addiu    $t3, $t2, 4     # adresa arr[j+1] = arr[j] + 4

    lw       $t0, 0($t2)    # $t0 = arr[j]
    lw       $t1, 0($t3)    # $t1 = arr[j+1]

    # if (arr[j] > arr[j+1]) swap
    slt      $t8, $t1, $t0   # $t8 = 1 ako arr[j+1] < arr[j]
    beq      $t8, $zero, no_swap

    sw       $t1, 0($t2)    # arr[j] = arr[j+1]
    sw       $t0, 0($t3)    # arr[j+1] = arr[j]
```

```

no_swap:
    addiu    $s1, $s1, 1    # j++
    j        inner_loop

inner_done:
    addiu    $s0, $s0, 1    # i++
    j        outer_loop

outer_done:
    lw        $s0, 4($sp)
    lw        $s1, 0($sp)
    addiu    $sp, $sp, 8
    jr        $ra

```

Z4. Faktorijal – iterativna verzija

Izračunati $n!$ iterativnom petljom.

C++

```

int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

```

Mapiranje: \$a0 = n, \$v0 = result, \$t0 = i, \$t1 = pomoćni registar za uslov petlje

MIPS

```

factorial:
    li        $v0, 1        # result = 1
    li        $t0, 2        # i = 2
fact_loop:
    slt       $t1, $a0, $t0  # $t1 = 1 ako n < i (tj. i > n)
    bne       $t1, $zero, fact_done  # ako i > n, gotovo

    mult      $v0, $t0       # HI:LO = result * i
    mflo      $v0           # result = donja 32 bita

    addiu     $t0, $t0, 1    # i++
    j         fact_loop

fact_done:
    jr        $ra

```

Z5. Linearno pretraživanje (Linear Search)

Pronaći indeks prvog pojavljivanja tražene vrijednosti u nizu. Ako nije pronađena, vrati -1.

C++

```
int linearSearch(int* arr, int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

Mapiranje: \$a0 = arr, \$a1 = n, \$a2 = target, \$v0 = povratna vrijednost, \$t0 = i, \$t1 = arr[i], \$t2 = pomoćni registar za uslov petlje

MIPS

```
linearSearch:
    li      $t0, 0          # i = 0
search_loop:
    slt     $t2, $t0, $a1    # i < n?
    beq     $t2, $zero, not_found

    sll     $t1, $t0, 2      # offset = i * 4
    addu    $t1, $a0, $t1    # adresa arr[i]
    lw      $t1, 0($t1)      # $t1 = arr[i]

    beq     $t1, $a2, found  # arr[i] == target?
    addiu   $t0, $t0, 1      # i++
    j       search_loop

found:
    addu    $v0, $t0, $zero  # return i
    jr      $ra

not_found:
    li      $v0, -1         # return -1
    jr      $ra
```

Z6. Obrtanje niza

Obrnuti redoslijed elemenata niza in-place.

C++

```
void reverseArray(int* arr, int n) {
    int left = 0, right = n - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}
```

Mapiranje: \$a0 = arr, \$a1 = n, \$s0 = left, \$s1 = right, \$t0 = arr[left], \$t1 = arr[right], \$t2 = pomoćni registar za uslov i adresu arr[left], \$t3 = pomoćni registar za adresu arr[right]

MIPS

```
reverseArray:
    addiu    $sp, $sp, -8
    sw       $s0, 4($sp)
    sw       $s1, 0($sp)

    li       $s0, 0           # left = 0
    addiu    $s1, $a1, -1     # right = n - 1

rev_loop:
    slt      $t2, $s0, $s1    # left < right?
    beq      $t2, $zero, rev_done

    # Adrese arr[left] i arr[right]
    sll      $t2, $s0, 2
    addu     $t2, $a0, $t2    # adresa arr[left]

    sll      $t3, $s1, 2
    addu     $t3, $a0, $t3    # adresa arr[right]

    lw       $t0, 0($t2)      # $t0 = arr[left]
    lw       $t1, 0($t3)      # $t1 = arr[right]

    sw       $t1, 0($t2)      # arr[left] = arr[right]
    sw       $t0, 0($t3)      # arr[right] = arr[left]

    addiu    $s0, $s0, 1      # left++
    addiu    $s1, $s1, -1     # right--
    j        rev_loop

rev_done:
    lw       $s0, 4($sp)
    lw       $s1, 0($sp)
    addiu    $sp, $sp, 8
    jr       $ra
```

Z7. Množenje matrica (Matrix Multiplication)

Množenje kvadratnih matrica dimenzije $N \times N$. Matrice su pohranjene u row-major order.

C++

```
void matMul(int* A, int* B, int* C, int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i * N + j] = 0;
            for (int k = 0; k < N; k++) {
                C[i * N + j] += A[i * N + k] * B[k * N + j];
            }
        }
    }
}
```

Adresa elementa $M[i][j]$ u row-major order za matricu $N \times N$: $\text{adresa} = \text{bazna_adresa} + (i * N + j) * 4$.

Mapiranje: $\$a0 = A$, $\$a1 = B$, $\$a2 = C$, $\$a3 = N$, $\$s0 = i$, $\$s1 = j$, $\$s2 = k$, $\$t0 = A[i][k]$, $\$t1 = B[k][j]$, $\$t2 = \text{suma za } C[i][j]$, $\$t4, \$t5, \$t6, \$t9 = \text{pomoćni registri}$

MIPS

```
matMul:
    addiu    $sp, $sp, -16
    sw       $s0, 12($sp)
    sw       $s1, 8($sp)
    sw       $s2, 4($sp)
    sw       $s3, 0($sp)

    addu     $s3, $a3, $zero    # s3 = N
    li       $s0, 0            # i = 0

i_loop:
    slt      $t9, $s0, $s3      # i < N ?
    beq      $t9, $zero, i_done

    li       $s1, 0            # j = 0

j_loop:
    slt      $t9, $s1, $s3      # j < N ?
    beq      $t9, $zero, j_done

    li       $t2, 0            # suma = 0
    li       $s2, 0            # k = 0

k_loop:
    slt      $t9, $s2, $s3      # k < N ?
    beq      $t9, $zero, k_done

    # A[i][k] = A + (i*N + k)*4
    mult     $s0, $s3
    mflo     $t4
    addu     $t4, $t4, $s2
    sll      $t4, $t4, 2
    addu     $t4, $a0, $t4
    lw       $t0, 0($t4)        # t0 = A[i][k]

    # B[k][j] = B + (k*N + j)*4
```

```

        mult    $s2, $s3
        mflo    $t5
        addu    $t5, $t5, $s1
        sll     $t5, $t5, 2
        addu    $t5, $a1, $t5
        lw      $t1, 0($t5)      # t1 = B[k][j]

        # suma += A[i][k] * B[k][j]
        mult    $t0, $t1
        mflo    $t6
        addu    $t2, $t2, $t6

        addiu    $s2, $s2, 1      # k++
        j        k_loop

k_done:
        # C[i][j] = suma
        mult    $s0, $s3
        mflo    $t4
        addu    $t4, $t4, $s1
        sll     $t4, $t4, 2
        addu    $t4, $a2, $t4
        sw      $t2, 0($t4)

        addiu    $s1, $s1, 1      # j++
        j        j_loop

j_done:
        addiu    $s0, $s0, 1      # i++
        j        i_loop

i_done:
        lw      $s0, 12($sp)
        lw      $s1, 8($sp)
        lw      $s2, 4($sp)
        lw      $s3, 0($sp)
        addiu    $sp, $sp, 16
        jr      $ra

```